

Étude d'architecture logicielle sur Vinted

1. Présentation du contexte

1.a Description de Vinted

Vinted est une application qui permet à des particuliers d'acheter et de vendre des vêtements et des objets d'occasion. On peut l'utiliser sur téléphone ou sur ordinateur, dans plusieurs pays d'Europe. On ne sait pas exactement comment Vinted est construit : cette étude propose donc une architecture imaginée à partir de ce qu'on voit en utilisant l'application.

Qui utilise Vinted ? D'abord les membres, qui peuvent acheter et vendre avec le même compte. Ensuite les équipes de Vinted : le service client, qui aide en cas de problème comme un colis perdu, la modération, qui supprime les fausses annonces et les arnaques, et les équipes qui suivent l'activité du site. Enfin, les partenaires : les transporteurs et les services de paiement.

Comment ça marche ? Le vendeur met en ligne une annonce avec des photos, une description, la taille, la marque et le prix. L'acheteur cherche un article, peut le mettre en favori et proposer un autre prix par message. Il paie dans l'application, et Vinted garde l'argent tant que le colis n'est pas arrivé. Le vendeur imprime une étiquette et dépose le colis. L'acheteur le récupère en point relais, en casier ou chez lui. Quand il confirme la réception, le vendeur reçoit l'argent dans son porte-monnaie Vinted et peut le transférer sur son compte bancaire. L'acheteur peut ensuite laisser un avis.

Les contraintes. Chaque article est unique, donc il ne doit jamais être vendu deux fois, même si deux personnes l'achètent en même temps. L'application gère de l'argent et des informations personnelles : elle doit être bien sécurisée et respecter le RGPD. Elle dépend d'autres entreprises pour les paiements et les livraisons. Elle doit aussi stocker beaucoup de photos, marcher dans plusieurs pays et rester rapide sur téléphone. Enfin, pour que les gens se fassent confiance, les avis, la modération et la protection des acheteurs sont indispensables.

1.b Principaux enjeux

Vinted est très utilisée et qui gère de l'argent et des données privées. Plusieurs enjeux doivent donc guider son architecture :

- **Montée en charge** : l'appli doit continuer à bien marcher quand des millions de personnes s'en servent en même temps, par exemple un dimanche soir.
- **Disponibilité** : l'appli doit être accessible tout le temps. Si elle tombe en panne, les gens ne peuvent plus ni acheter ni vendre, et Vinted perd de l'argent.
- **Sécurité** : il faut protéger l'argent et les informations privées, et empêcher les arnaques.
- **Maintenabilité** : le code doit rester facile à corriger et à modifier, même après des années.
- **Évolutivité** : on doit pouvoir ajouter facilement de nouvelles fonctionnalités, de nouveaux pays ou de nouveaux transporteurs.
- **Travail en équipe** : de nombreux développeurs travaillent en même temps sur l'appli, sans se gêner les uns les autres.

2. Choix d'architecture

(2.a & 2.b) Architecture retenue et justification

Vinted est construit comme un **monolithe modulaire**, une seule application, découpée en modules indépendants qui communiquent entre eux de façon encadrée.

Les modules Paiement et Livraison utilisent une **architecture hexagonale** : ils passent par une « prise » standard, ce qui permet de brancher ou changer un prestataire (PayPal, un transporteur...) sans toucher au reste de l'application.

Le monolithe modulaire est simple à lancer et à faire évoluer, tout en gardant un code bien rangé. Chaque module pourra plus tard devenir un service séparé si besoin.

- **Monolithe classique** : simple au départ, mais le code devient vite un gros bloc où tout est mélangé, difficile à modifier à plusieurs.
- **Microservices dès le départ** : trop complexe et coûteux pour une application qui démarre, avec beaucoup d'outils et de pannes possibles entre les services.

2.c Schéma d'architecture

Le schéma montre les différentes parties de l'appli et comment elles fonctionnent ensemble.

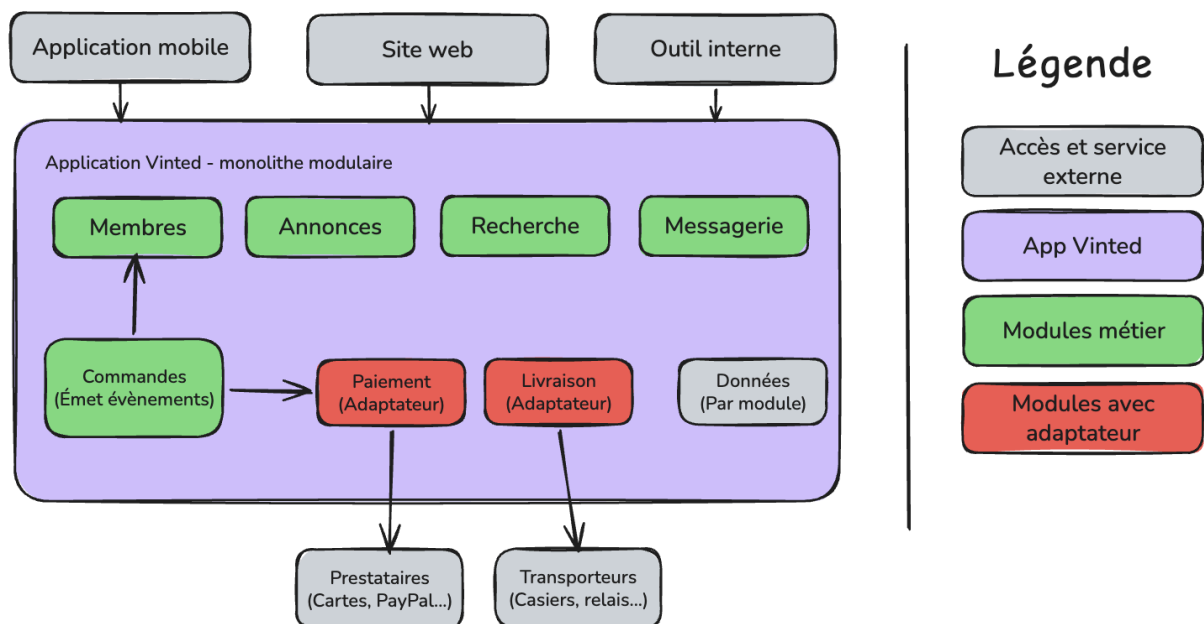


Schéma fait sur [excalidraw](#)

3. Organisation du code

3.a Structuration du projet

Je donnerais à chaque module son propre dossier, organisé de la même manière en quatre couches. La première recevrait les demandes de l'application, la deuxième contiendrait les règles métier, la troisième gérerait la base de données et la dernière communiquerait avec les services externes.

(3.b & 3.c) Structuration SOLID et éviter la dette technique

La première décision consiste à ce que chaque module garde ses propres données. Un module qui a besoin d'une information la demande à un autre, au lieu d'aller la chercher lui-même, ce qui évite qu'un changement casse toute l'application. La seconde concerne les services externes : le code ne parlerait jamais directement à PayPal ou à un transporteur, il passerait par un adaptateur, qu'il suffirait de remplacer en cas de changement de prestataire.

4. Patterns et conception

(4.a & 4.b)

Pour cette architecture, j'ai retenu deux design patterns : **L'Adaptateur et l'Observateur.**

Vinted travaille avec plusieurs prestataires de paiement et de nombreux transporteurs, qui ont chacun leur propre fonctionnement. Sans solution adaptée, le code serait rempli de cas particuliers.

L'Adaptateur règle ce problème : je définis une façon unique de « créer un paiement » ou de « générer une étiquette », et chaque prestataire a son adaptateur qui traduit cette demande dans son propre langage. Ajouter un transporteur dans un nouveau pays revient alors à écrire un nouvel adaptateur.

L'Observateur répond à un autre besoin. Quand l'acheteur confirme la réception d'un colis, l'argent doit partir dans le porte-monnaie du vendeur, une notification doit lui être envoyée et l'acheteur peut laisser un avis. Plutôt que de lier tous ces modules entre eux, la commande annonce simplement « réception confirmée », et chaque module concerné réagit de son côté.

Une nouvelle réaction pourra ainsi être ajoutée plus tard sans toucher au module «Commandes».

5. Qualité du développement

5.a Pratiques de qualité

Pour que plusieurs équipes puissent travailler sur le même projet sans perdre en qualité, je mettrais en place quelques règles simples :

- **Git** : chaque nouvelle fonctionnalité est développée sur une branche à part, puis ajoutée au projet principal une fois terminée.
- **Code review** : aucun code n'est ajouté sans avoir été relu et validé par au moins un autre développeur.
- **Conventions** : toute l'équipe suit les mêmes règles d'écriture (noms, mise en forme), vérifiées automatiquement par un outil.
- **Tests automatisés** : ils vérifient à chaque modification que rien n'a été cassé.
- **CI (intégration continue)** : à chaque envoi de code, les tests et les vérifications se lancent tout seuls. Si quelque chose échoue, le code est bloqué.
- **Documentation** : chaque module a une courte fiche expliquant son rôle et comment l'utiliser.
- **Gestion des dépendances** : les bibliothèques externes sont limitées au nécessaire et mises à jour régulièrement, surtout pour corriger des failles de sécurité.

5.b Stratégie de tests

Pour les tests, je m'appuie sur le principe de la pyramide : beaucoup de petits tests rapides à la base, et seulement quelques tests complets au sommet. Cette approche permet de détecter la plupart des erreurs très tôt, sans ralentir le travail des équipes.

- **Tests unitaires** : ils vérifient une petite règle isolée, par exemple que le calcul des frais de protection est juste.
- **Tests d'intégration** : ils vérifient que plusieurs parties fonctionnent ensemble, par exemple que la confirmation d'une commande crédite bien le porte-monnaie du vendeur.
- **Tests end-to-end** : ils simulent un vrai utilisateur, de la recherche d'un article jusqu'au paiement, uniquement sur les parcours les plus importants.

6. Evolution de l'architecture

a. Le scénario

Avec quelques centaines d'utilisateurs, une seule application et une seule base de données suffisent. En passant à plusieurs centaines de milliers d'utilisateurs, les recherches, les photos et les messages se multiplient, et certaines parties de l'application commencent à ralentir.

b. Ce qui devrait évoluer

Je ferais évoluer l'architecture progressivement, en commençant par les parties les plus sollicitées :

- **L'application** : je la lancerais en plusieurs exemplaires, avec un répartiteur de charge qui envoie chaque utilisateur vers celui qui est le moins occupé.
- **Les photos** : elles seraient stockées dans un service cloud et diffusées par un CDN, qui les rapproche des utilisateurs pour qu'elles s'affichent plus vite.
- **La recherche** : ce module deviendrait un service séparé, avec un moteur de recherche spécialisé, car c'est la fonctionnalité la plus utilisée.
- **La base de données** : j'ajouterais des copies en lecture seule et un cache pour les informations consultées très souvent, comme les annonces populaires.

Certains choix faits au départ aident à grandir. Comme les modules sont déjà séparés, je peux en sortir un de l'appli sans tout refaire. Et grâce au pattern "Observateur", certaines tâches pourront se faire en arrière-plan, sans faire attendre l'utilisateur.

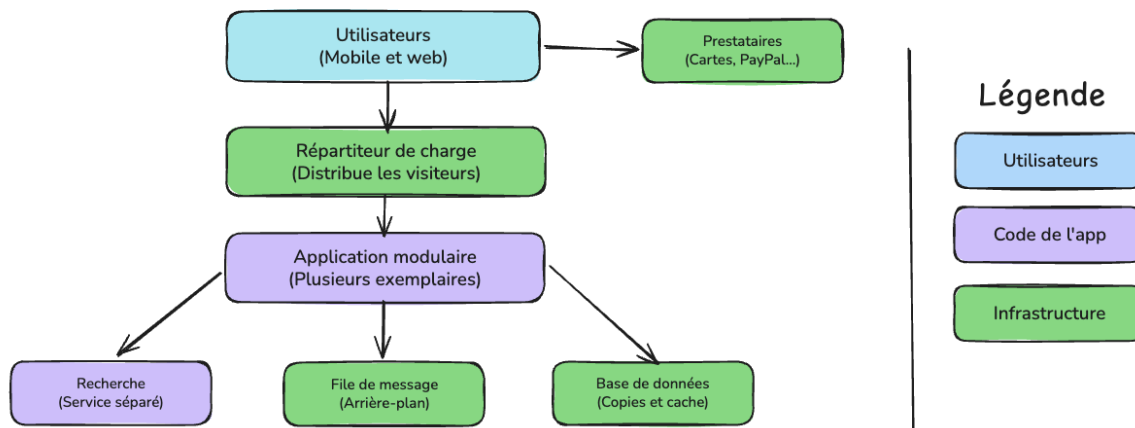


Tableau synthétisant les principales décisions d'architecture, leurs avantages et leurs limites :

<u>Décision</u>	<u>Avantages</u>	<u>Limites</u>
Monolithe modulaire	Simple à lancer et à déployer, code bien rangé par sujet, modules faciles à sortir plus tard	Toute l'application est déployée d'un bloc, une grosse panne peut tout arrêter
Architecture hexagonale et adaptateurs pour le paiement et la livraison	Changer ou ajouter un prestataire ne touche pas au reste du code	Demande plus de code au départ et un peu plus de réflexion
Données séparées par module	Une modification dans un module ne casse pas les autres	Récupérer des informations de plusieurs modules à la fois est plus long
Pattern Observateur	Ajout de nouvelles réactions sans modifier le module Commandes, prépare les files de messages	Le déroulement d'une action est plus difficile à suivre en cas de bug
Code review, CI et pyramide de tests	Moins de bugs en production, qualité homogène entre les équipes	Ralentit un peu la mise en ligne, les tests demandent du temps à écrire et à maintenir

Évolution progressive vers des services séparés	On ne complexifie que les parties qui en ont vraiment besoin, comme la recherche	Plus de services veut dire plus d'outils à gérer et de nouvelles sources de panne
--	--	---